

# COMP1549: Advanced Programming

Part a: Introduction to Components  
Part b: JavaBeans

Dr Markus Wolf

12<sup>th</sup> March, 2026 - Week 9

# Introduction to Components



# Definitions of Component

- ECOOP (European Conference on Object-Oriented Programming) 1996
  - “A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties”

# More Definitions

- Szyperski 1998 (first edition of Component Software)
  - “Software components are binary units of independent production, acquisition, and deployment that interact to form a functioning system”
- Grady Booch (one of the 3 amigos who gave us UML)
  - ❖ “A reusable software component is a logically cohesive, loosely coupled module that denotes a single abstraction”

# More Definitions

- Object-Oriented Systems Analysis and Design Using UML
  - “Component is a replaceable part of a system defined primarily in terms of the interfaces that it provides and the interfaces that it requires in order to operate.”
- The Unified Modelling Language Reference Manual
  - “A component represents a modular piece of a logical or physical system whose externally visible behaviour can be described much more concisely than its implementation. Components do not depend directly on other components but on interfaces that components support”

# More Definitions

- Meta Group
  - ❖ “Software components are defined as prefabricated, pretested, self-contained, reusable software modules – bundles of data and procedures – that perform specific functions”
- In the second edition of his book *Component Software*, Clemens Scyperski says that a component:
  - Is a unit of independent deployment
  - Is a unit of third-party composition
  - Has no (externally) observable state

# Hardware Analogy

- Similar idea to hardware components:



Photo by [Luke Hodde](#) on [Unsplash](#)

# Unit of Independent Deployment

- If a component is intended for reuse in a variety of contexts, it needs to be independently deployable
- Can not be partially deployed
- Requires that it encapsulates its constituent features
- Requires it is separated from its environment and other components
  - Could specify required interfaces

# Unit of Third-Party Composition

---

- To achieve maximum reusability, it should be possible for third-parties to use components
- Third-party may not have access to construction details – it's a black-box
- Requires clear specification of what it requires and provides

# No (Externally) Observable State

- It should be possible to exchange a component with another that fulfils the same contract
- Requires a component not to have observable state
  - There is a difference between a component and an instance of a component – think class and object
  - Objects that make up a component instance can be injected with state – e.g. a button is a component which is wired to your application and can have some state, such as a position, text, etc.

# What are components then?

- Present a simple interface to clients
- Have a well-defined role
  - Need to be general enough to be used in a variety of contexts – granularity of a component
- Represent an individual unit of deployment
  - Though self-contained, they might require deployment within a service-providing framework or container
  - May contain one or more classes
- Components are for composition
  - May consist of smaller components
  - May be themselves part of a larger component

# What are components then?

- Designed for reuse
- Could use standards to enhance their interoperability
  - May depend on the target domain
- Typically provided as black-box implementations
  - Require well-defined interfaces and documentation
- Could be custom-made or purchased

# Benefits of Components

- Reuse, reuse, reuse...
- Provides an alternative to bespoke and standard software
  - Bespoke software
    - ❖ Tailored to exact needs
    - ❖ But expensive to develop, lack of expertise leading to missing functionality or software not working
  - Standard software
    - ❖ Cheaper
    - ❖ Parameterised to provide the closest possible solution, but may not be fitting or impose changes on the business

# Benefits of Components

- Combine existing components to form tailored systems
  - Components could be purchased, available for free or developed for reuse within the same company
  - Could be a combination of bespoke software with existing components
  - Cost-efficient, as components may be reused
  - Has to be well-defined and deployable
- Breaking down an application into smaller pieces (components) helps handling complexity

# Disadvantages of Components

- Overhead in development
  - Complexity
  - Time
  - Cost
- Version hell
  - What happens if we want to change a component which is already used by clients?
- YAGNI (you ain't gonna need it)
  - Adding functionality that isn't needed

# Importance of “Good” Contracts

- Contractually specified interfaces decouple the client and provider
- Self-standing interfaces enable:
  - Many possible clients using the interface
  - Many possible providers implementing the interface
- Don't demand too much or too little from:
  - Client – feasible preconditions
  - Provider – feasible postconditions
- Specify non-functional requirements
  - E.g. performance requirements
- History specification (e.g. permissible sequence of operations)

# Deploying a Component

- A component is for composition and needs to be in an environment in which to operate
- Components are hosted by containers, containing applications or frameworks
- These can provide services to the components
  - Instantiation and destruction
  - Security
  - Visual display
  - Distribution
  - Others

# Interfaces and Components

- An interface is an operational contract. It guarantees implementers will be able to fulfil named operations (methods)
- Define abstract supertypes
  - Whatever implements the interface is a subtype
    - ❖ E.g. Dog class implements Animal interface – so Dog is a subtype of Animal
- Components which implement the same interface are interchangeable
- Interfaces should be immutable

# Default Methods in Interfaces

- Also called “Defender Methods”
- Introduced to Java in version 8
- Possible to add new methods to interfaces without breaking existing implementations
- How is this different from an abstract class?
  - Conceptually they are different
  - Abstract classes can have constructors
    - ❖ Abstract classes get instantiated (but not by us)
  - Abstract class can have state
    - ❖ Interfaces can only have static variables

# Default Method Example

- Consider the following interface and its implementation

```
public interface Writeable {  
    public void write(String text) ;  
}
```

What if we want to  
add another  
method later?

```
public class MyWriter implements Writeable {  
  
    @Override  
    public void write(String text) {  
        System.out.println("Writing the following: " + text);  
    }  
  
}
```

# Default Method Example

- This would break the existing *MyWriter* class:

```
public interface Writeable {  
    public void write(String text);  
    public void writeGerman(String text);  
}
```

- This would be fine:

```
public interface Writeable {  
    public void write(String text);  
  
    default public void writeGerman(String text) {  
        System.out.println("Schreibe folgendes: " + text);  
    };  
}
```

# Multiple Inheritance

- Java doesn't allow multiple inheritance of classes, but allows implementing multiple interfaces
  - If two default methods with the same method signature are inherited you can a compiler error
    - You need to provide your own implementation of the default method
      - ❖ In there you can still make a call to an inherited default method
- ```
@Override
public void writeGerman(String text) {
    Writeable.super.writeGerman(text);
}
```

# Late Binding

---

- One of the advantages of interfaces in a component system is that you don't need the implementation to compile and maintain your code
- The binding of the implementation code to the actual interface can occur not at compile time, but at runtime
- This is known as late binding

# Late Binding

---

- Possible to write a component that is dependent on another component without the two components actually meeting (until runtime deployment)
- An alternative supplier could support the same interface and provide a better (or cheaper) component implementation

# Examples of Industry Components

- Java
  - JavaBeans
  - Java Server Faces (JSF) - for web GUIs
- Microsoft
  - DCOM (Component Object Model)
  - .NET components
- Web Services (REST and SOAP)
- Unity Game Object Components

# Java Module

- Java 9 introduced the concepts of Modules
  - A module is a set of packages
- Reusable group of related packages, resources (e.g. media files) and a module descriptor
- Enforces strong encapsulation - prevents invocation of private (concealed) members through reflection
- Packages in a module fall into two categories:
  - **Exported Packages** - visible to the outside
  - **Concealed Packages** – can only be used from within the module

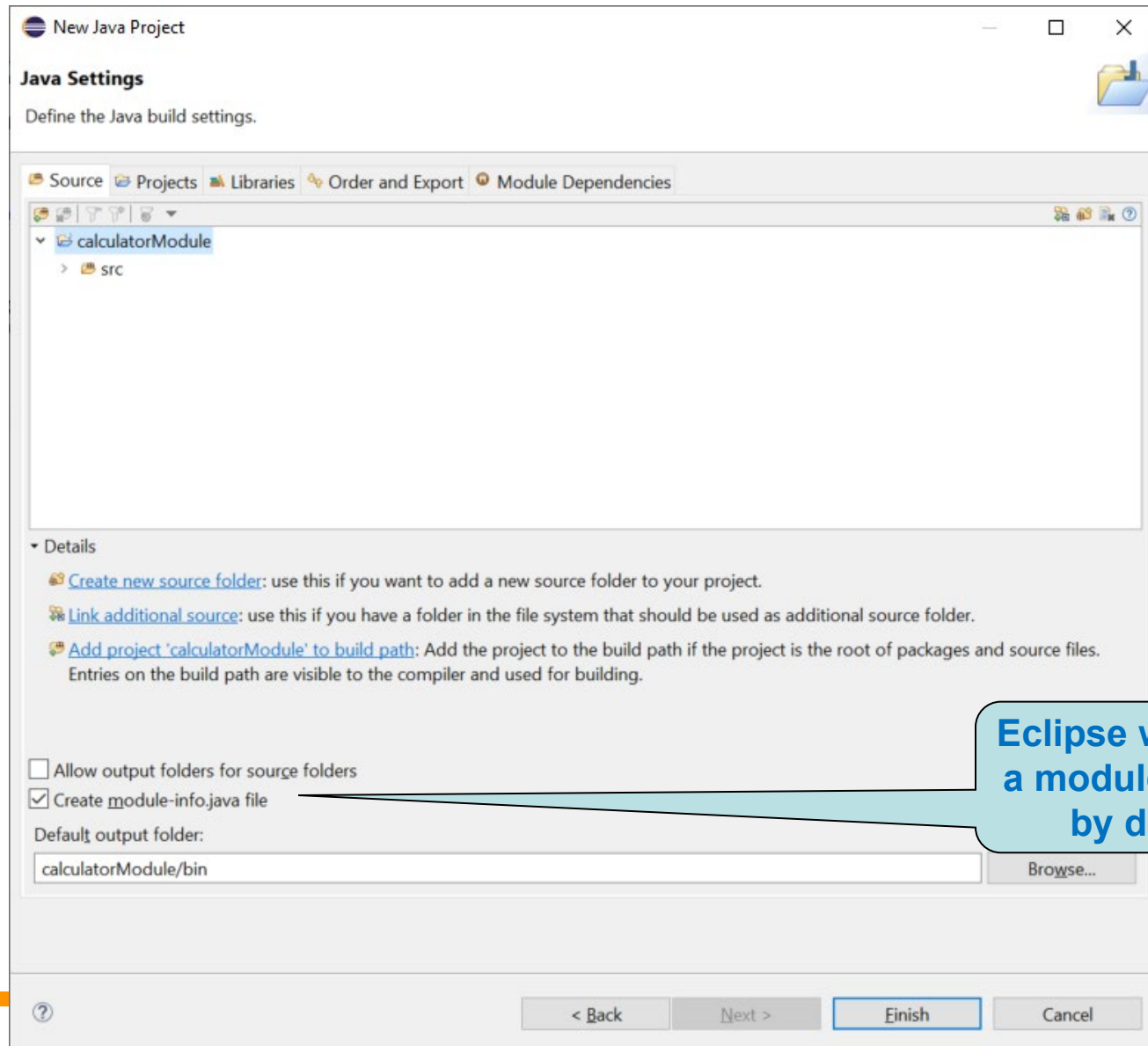
# Java Module

- A module contains a *module descriptor*:
  - Name
  - Dependencies (what it requires)
  - Public Packages (what the module exports and makes visible to others)
  - Services Offered (can provide service implementations)
  - Services Consumed (can consume services)
  - Reflection Permissions (can explicitly allow reflection – can't prevent seeing the content, but can prevent invocation)

# Some Key Modules

- **java.base** contains key packages (APIs), such as `java.lang`, `java.io`, `java.util`, `java.math`, `java.net`, `java.time`, etc.
- **java.desktop** contains AWT and Swing packages for creating user interfaces
- **java.sql** defines the JDBC API for working with relational databases
- **java.xml** defines the Java APIs for working with XML
- Many others...

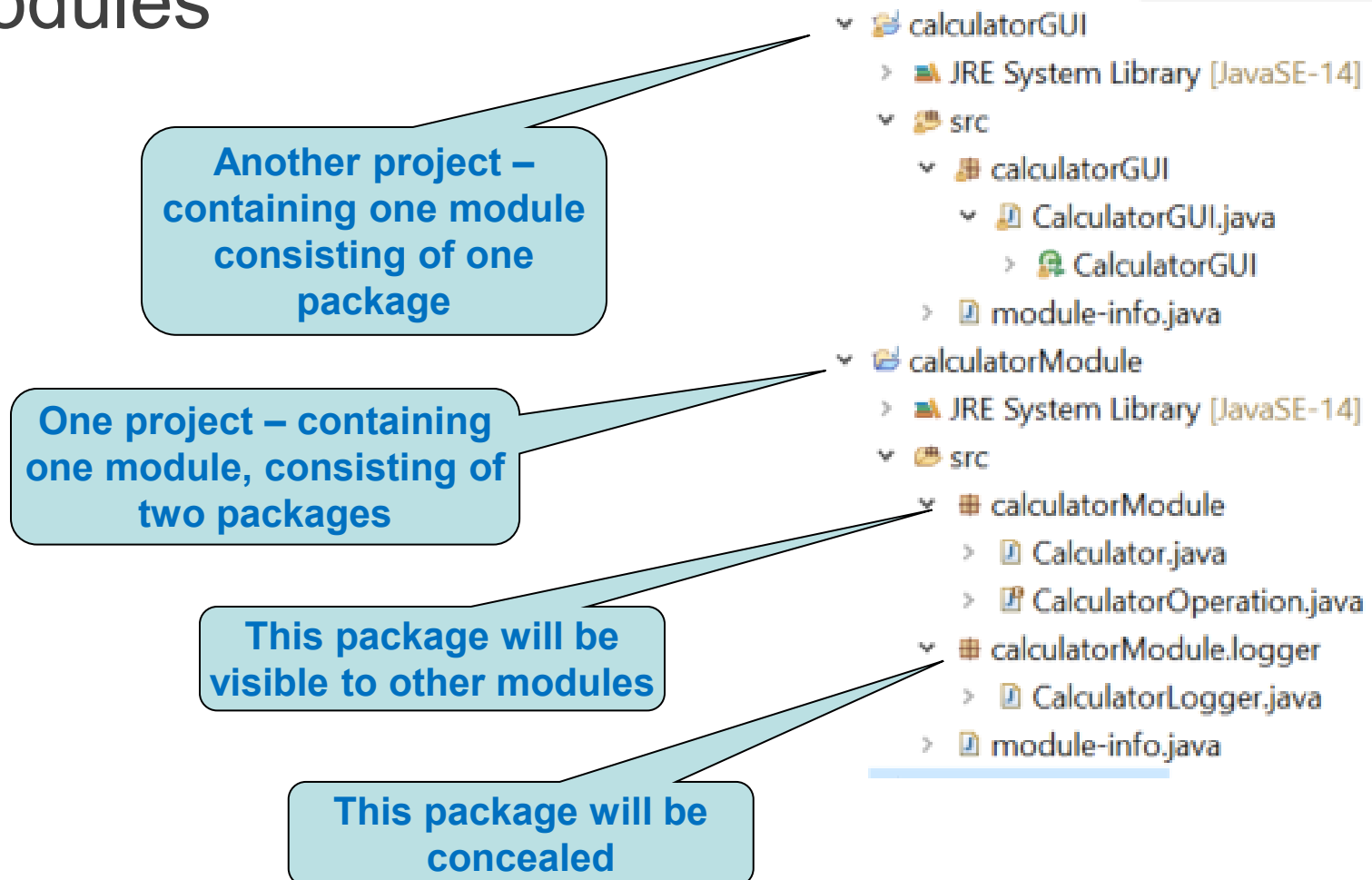
# Java Modules in Eclipse



Eclipse will create  
a module-info file  
by default

# Java Module Example

- Let's have a look and an example using modules

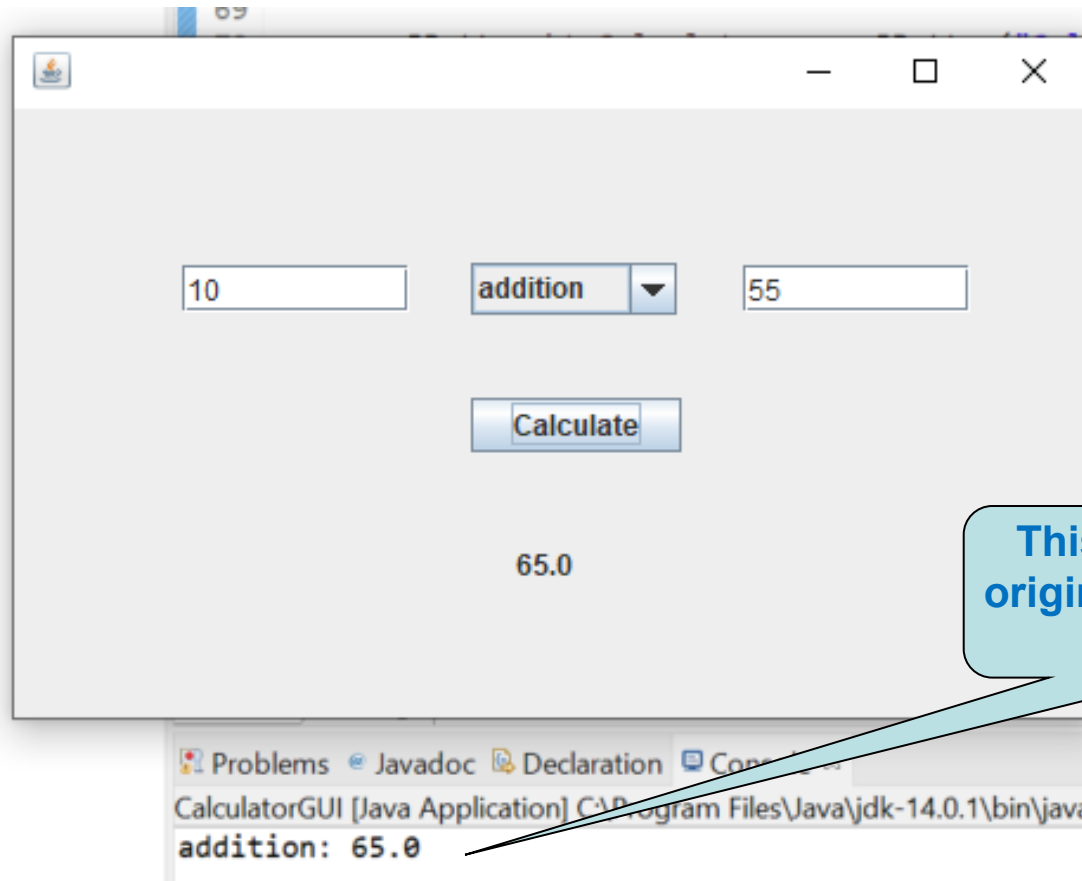


# Java Module Example

- CalculatorModule Project – this contains code to perform four simple operations on two numbers
  - calculatorModule package – contains a class to perform the operations and an enumeration listing the four operations
    - ❖ This is exported (visible)
  - calculatorModule.logger – contains a class that prints a log of any results calculated
    - ❖ This is concealed and can't be seen from the outside
- CalculatorGUI Project – this contains a desktop user interface application (JFrame)
  - Has a dependency on the CalculatorModule

# Java Module Example - GUI

- This is what the application looks like when running



This print statement  
originates in the logger  
class

# calculatorModule Package

```
public enum CalculatorOperation {  
    addition, subtraction, division, multiplication  
}
```

An enumeration of the operations

```
public class Calculator {  
    public float add(float num1, float num2) {  
        float result = num1 + num2;  
        CalculatorLogger.LogResult(result,  
                                   CalculatorOperation.addition);  
        return result;  
    }  
}
```

A class which implements a method for each operation

Calls the logger class to log the result

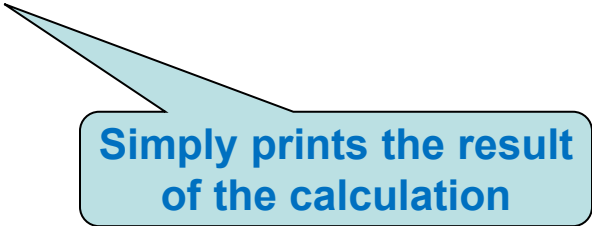
```
public float subtract(float num1, float num2) {  
    ...  
}
```

...

The remaining methods are almost identical to the add method

# calculatorModule.logger Package

```
public class CalculatorLogger {  
  
    public static void LogResult(float result,  
                                CalculatorOperation operation) {  
  
        System.out.println(operation + ": " + result);  
  
    }  
}
```



Simply prints the result  
of the calculation

# CalculatorModule Descriptor

- This is what the module-info.java file looks like

```
module calculatorModule {  
    requires java.base;  
    exports calculatorModule;  
}
```

Declares the name of  
the module

States that our module  
needs to use the  
java.base module –  
this could have been  
omitted as it is  
included by default

This means that only the calculatorModule  
package is exported (made visible), the  
calculatorModule.logger package is  
concealed

# CalculatorGUI Descriptor

- Here is the module-info.java file of the GUI project

```
module calculatorGUI {  
    requires java.desktop;  
    requires calculatorModule;  
}
```

java.base is added by default

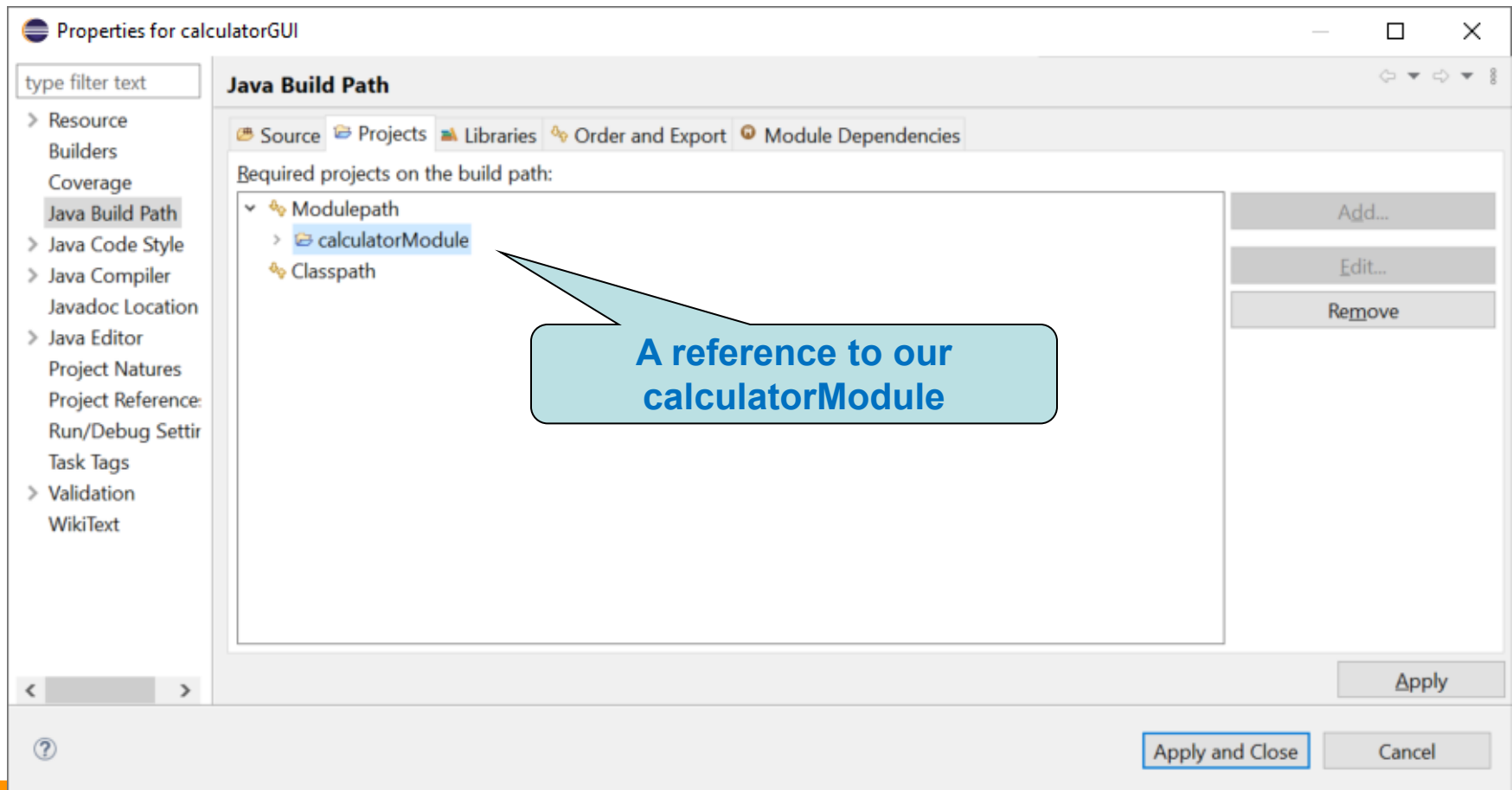
We will create a Swing GUI, so need this module

This module uses the calculatorModule we created

In addition to adding the requires statement, we also need to add the module to the build path

# Java Build Path

- To add a module to the build path, we go to the project properties in Eclipse



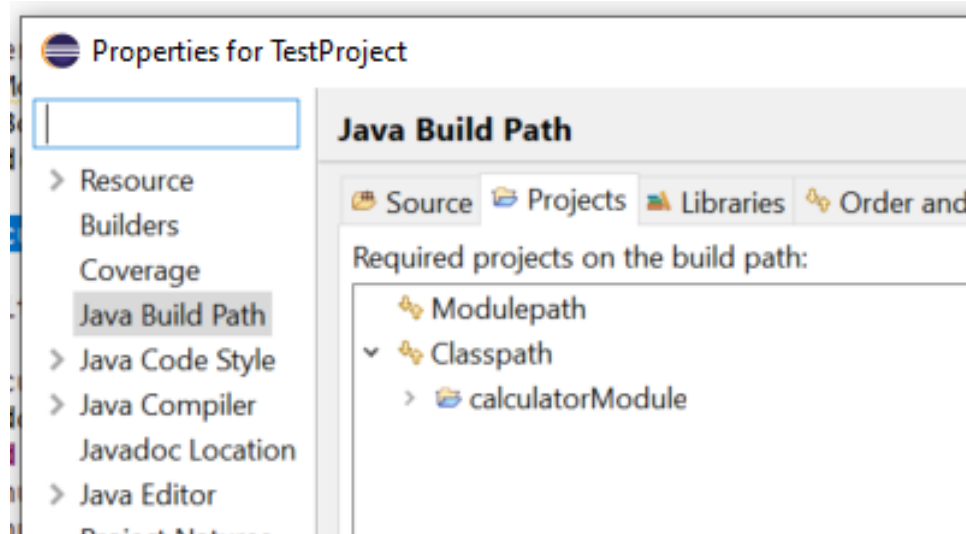
# calculatorGUI Class

- This excerpt shows the button click handling

```
Calculator calculator = new Calculator();  
btnCalculate.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        float num1 = Float.parseFloat(txtNum1.getText());  
        float num2 = Float.parseFloat(txtNum2.getText());  
        CalculatorOperation operation =  
            ((CalculatorOperation) cbOperator.getSelectedItem());  
        switch(operation) {  
            case addition:  
                lblResult.setText(((Float) calculator.add(num1,  
                    num2)).toString());  
                break;  
            case subtraction:  
                lblResult.setText(((Float) calculator.subtract(num1,  
                    num2)).toString());  
                break;  
            ...  
        }  
    }  
});
```

# Java Module Access

- Can a module package be accessed from a project which doesn't contain a module-info file?
  - Yes, you just need to ensure that it is on the build classpath



# JavaBeans

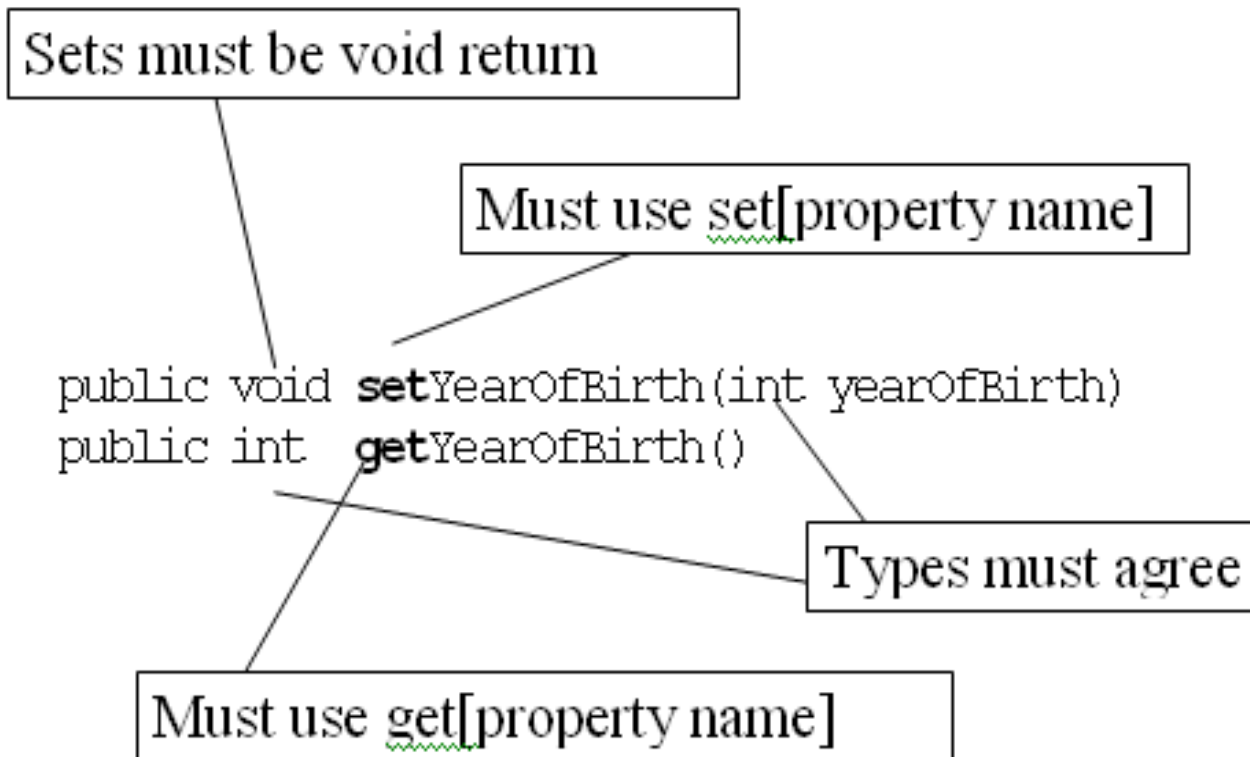


# JavaBeans

- According to the JavaBeans Specification:
  - “The goal of the JavaBeans APIs is to define a *software component model* for Java”
  - “A *Java Bean* is a reusable software component that can be manipulated visually in a builder tool.”
    - ❖ This definition shows that the initial impetus of this model was to allow certain features in development environments to be standardised

# Naming Convention

- Persistent Properties (encapsulated member variables) – prefixed get and set



# Indexed Properties

- If properties have a one to many relationship with the host object they must be exposed as an array and have appropriate index methods

```
public void setChild(Child [] children)
public void setChild(int index , Child child)
public Child[] getChild ()
public Child getChild(int index)
```

# Events

---

- Beans may publish that they generate events by conforming to the java event model
- They must have a means for listeners to add and remove themselves to the bean object

```
void addEventListener(java.util.EventListener lis)  
void removeEventListener(java.util.EventListener lis)
```

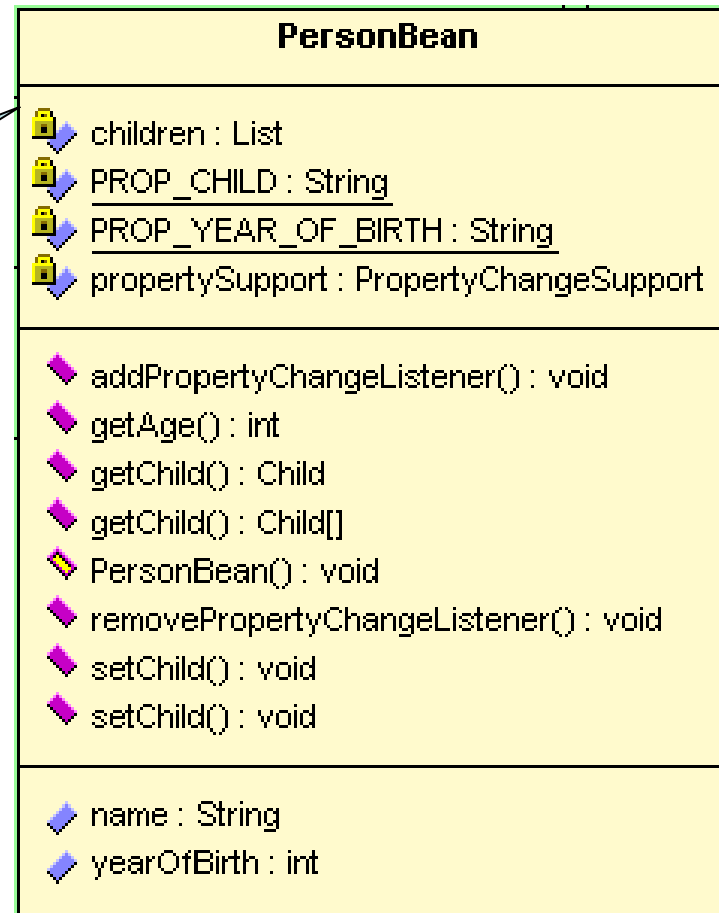
# Bound Properties

---

- When bound properties are changed a property change event is fired by the bean
  - This calls the `propertyChange()` method on all the registered listeners
  - All listeners will thus be notified of the fact that the particular property on the bean has changed

# A Bean Example

Note that the children variable is a List, but is exposed through the property as an array, to conform with the JavaBeans standard

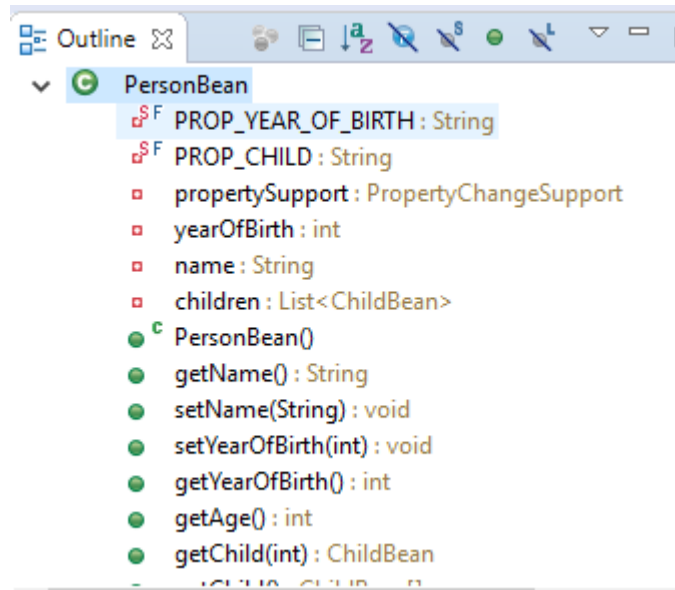


# Why Bother?

- When you implement the beans specification, it becomes possible for an IDE (Integrated Development Environment) to **introspect** on your beans and discover the name and types of properties you as a developer wish to make public
- It then becomes possible to visually drag a component representation in a form editor and set its properties just like you might do with a JButton (In fact a JButton is a JavaBean).

# Tools

## ● Eclipse view of a bean



# BeanInfo

- Developers may want to use the naming strategy associated with beans throughout their class but not necessarily expose all bean methods
- For this reason the Bean framework was extended with the BeanInfo in which you could restrict and extend the manner in which your object was exposed as a bean
- You could even give it an icon

# BeanInfo

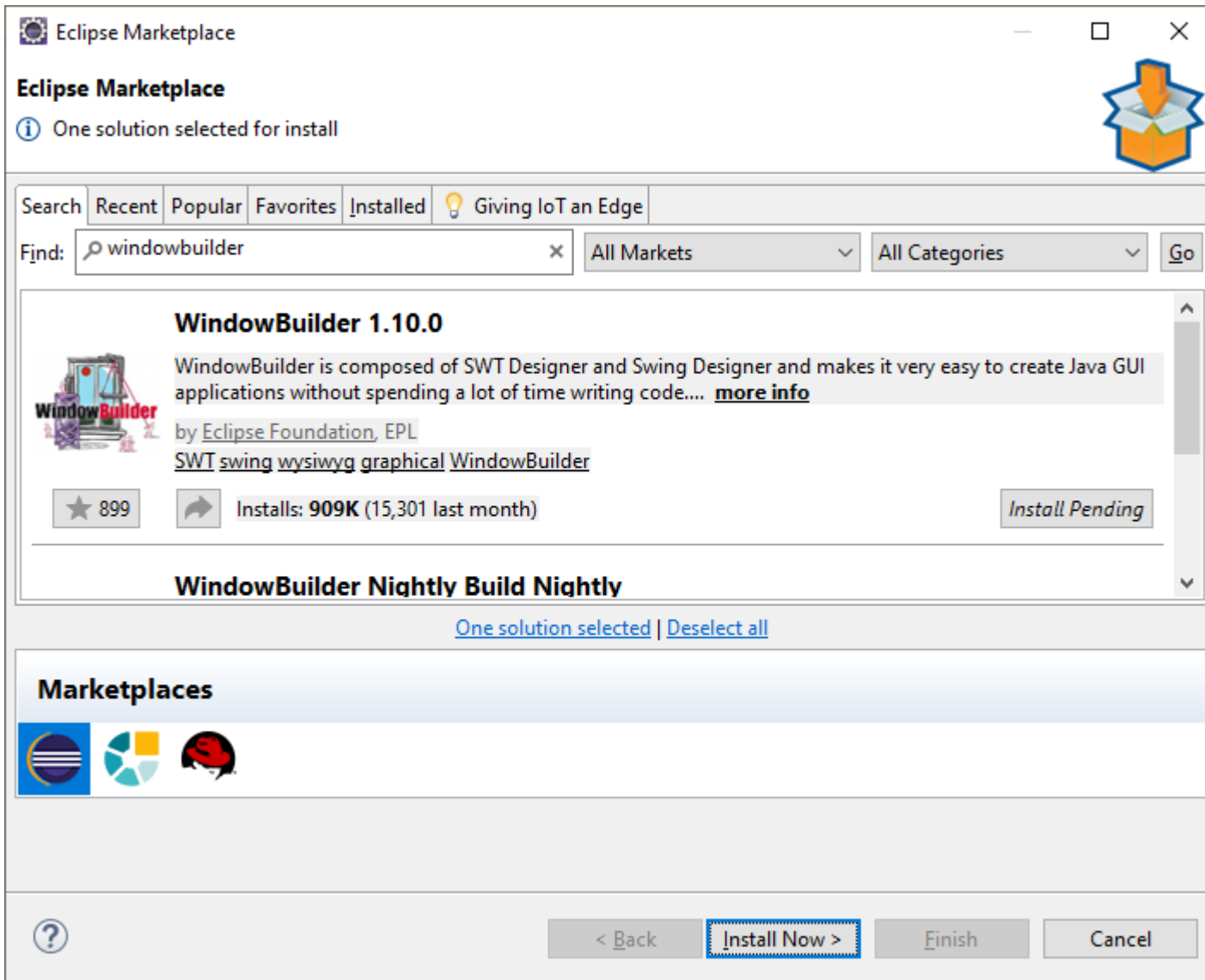
- Icons can be associated with Java Beans – so you get a pretty picture to represent your bean in toolboxes, toolbars, etc.
- In this case we have an image of a person 
- Some IDEs have a BeanInfo editor (e.g. NetBeans), but unfortunately Eclipse does not

# Visual Designer

---

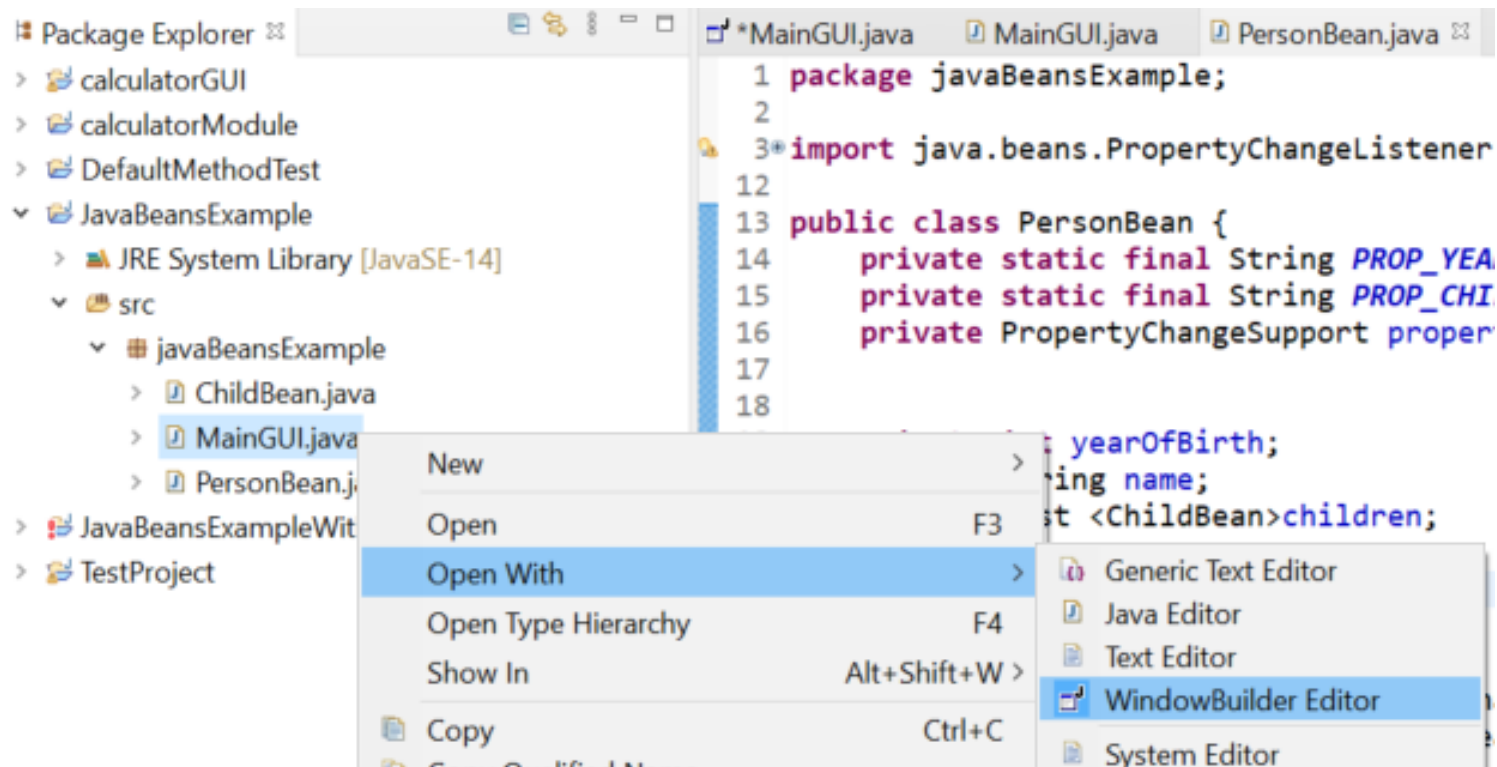
- There is an Eclipse plugin called WindowBuilder which provides a visual designer for AWT and Swing
- To install plugins you can search the Eclipse Market Place or go to the Help menu and select Install New Software

# Install a Plugin



# Using the Visual Designer

- The default editor is text-based, to use the visual designer select WindowBuilder Editor



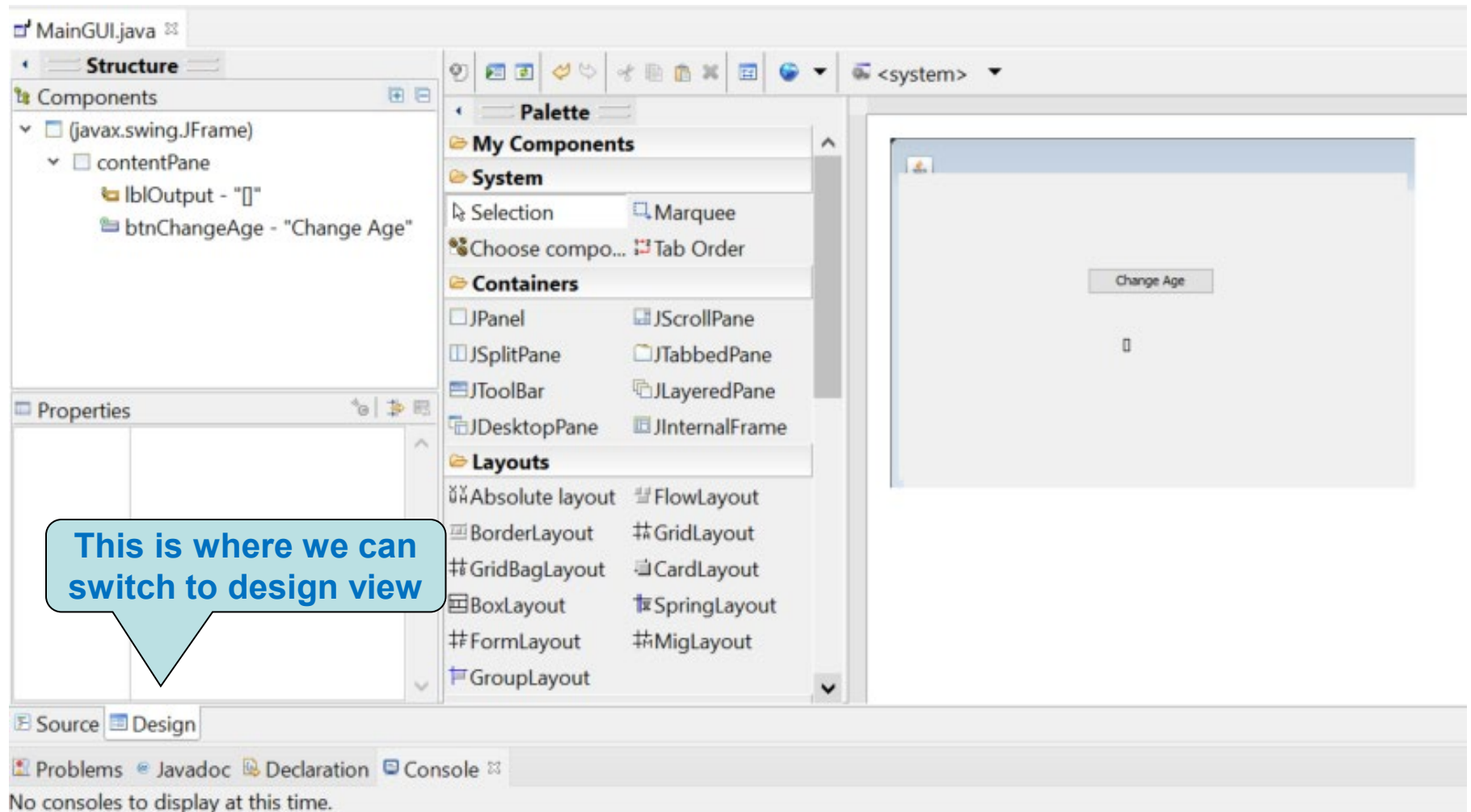
# Form that will use the PersonBean

---

- This is a very simple form with a button and a label
- When we press the button the age of the person is decremented by one (year of birth is incremented by one) and the name and age are displayed
- We want to be able to drop the bean on the form and set the name and year of birth through the IDE

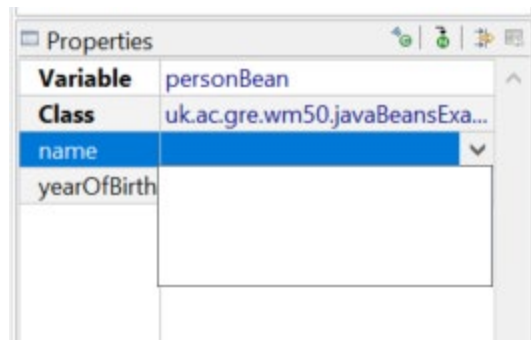
# The GUI

- It just contains a button and a label



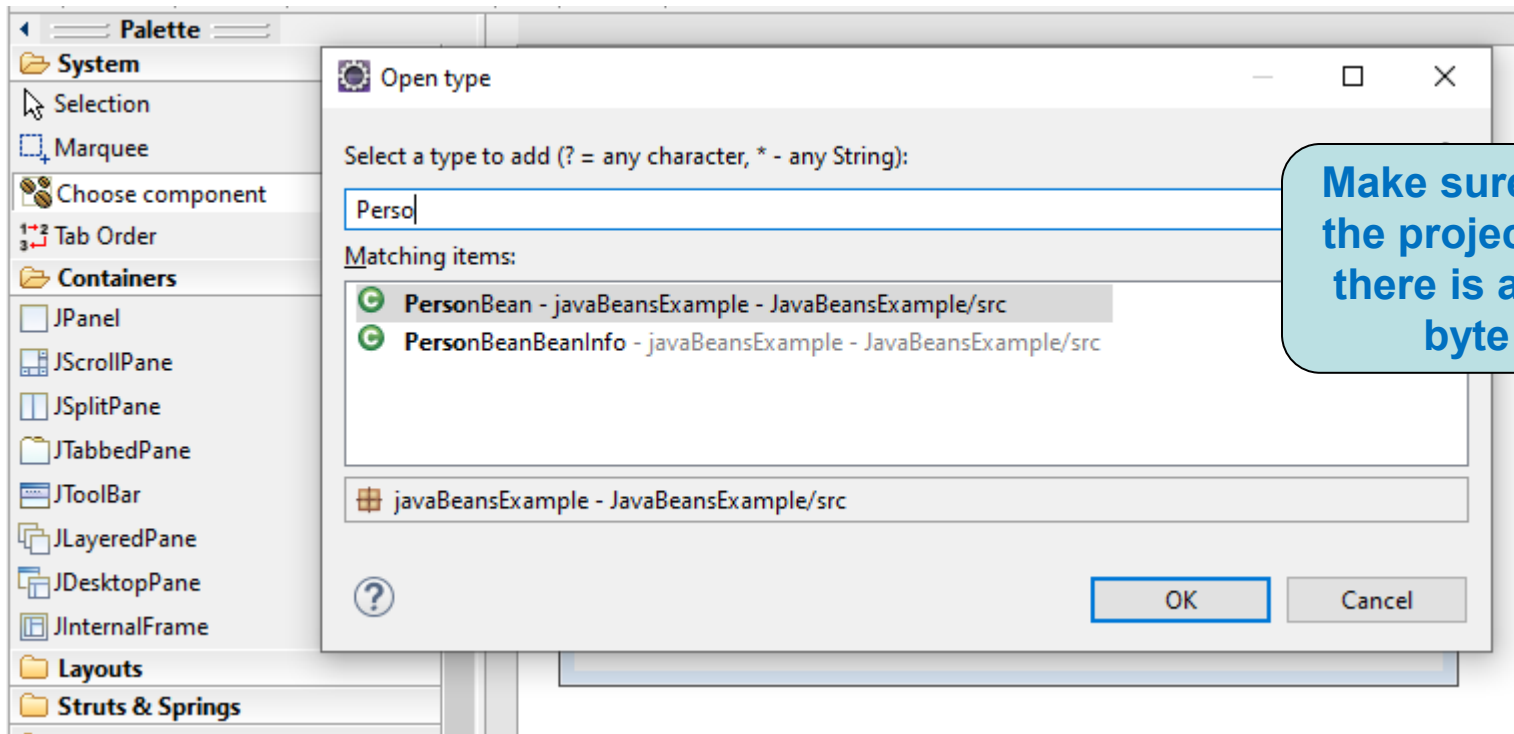
# Bug in WindowBuilder

- The BeanInfo file should be optional, but in the latest version of WindowBuilder, you won't be able to edit JavaBean properties unless you have a BeanInfo file
  - It also has to be compiled before the editing works
  - [https://bugs.eclipse.org/bugs/show\\_bug.cgi?id=565979](https://bugs.eclipse.org/bugs/show_bug.cgi?id=565979)



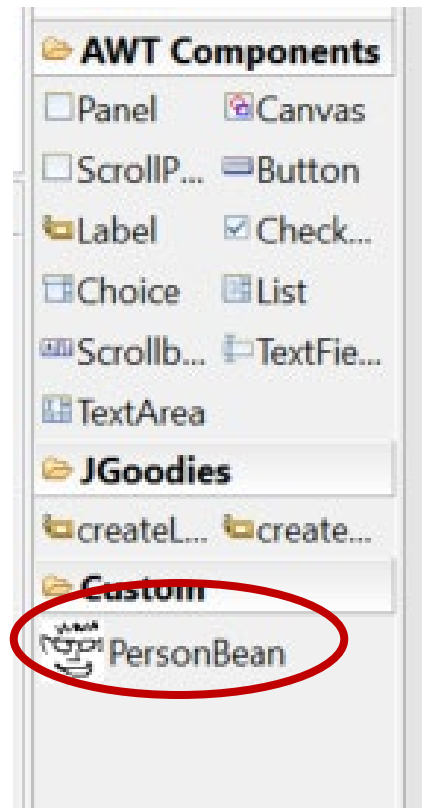
# Adding the PersonBean to the Palette

- Click on the *Choose component* item in the Palette



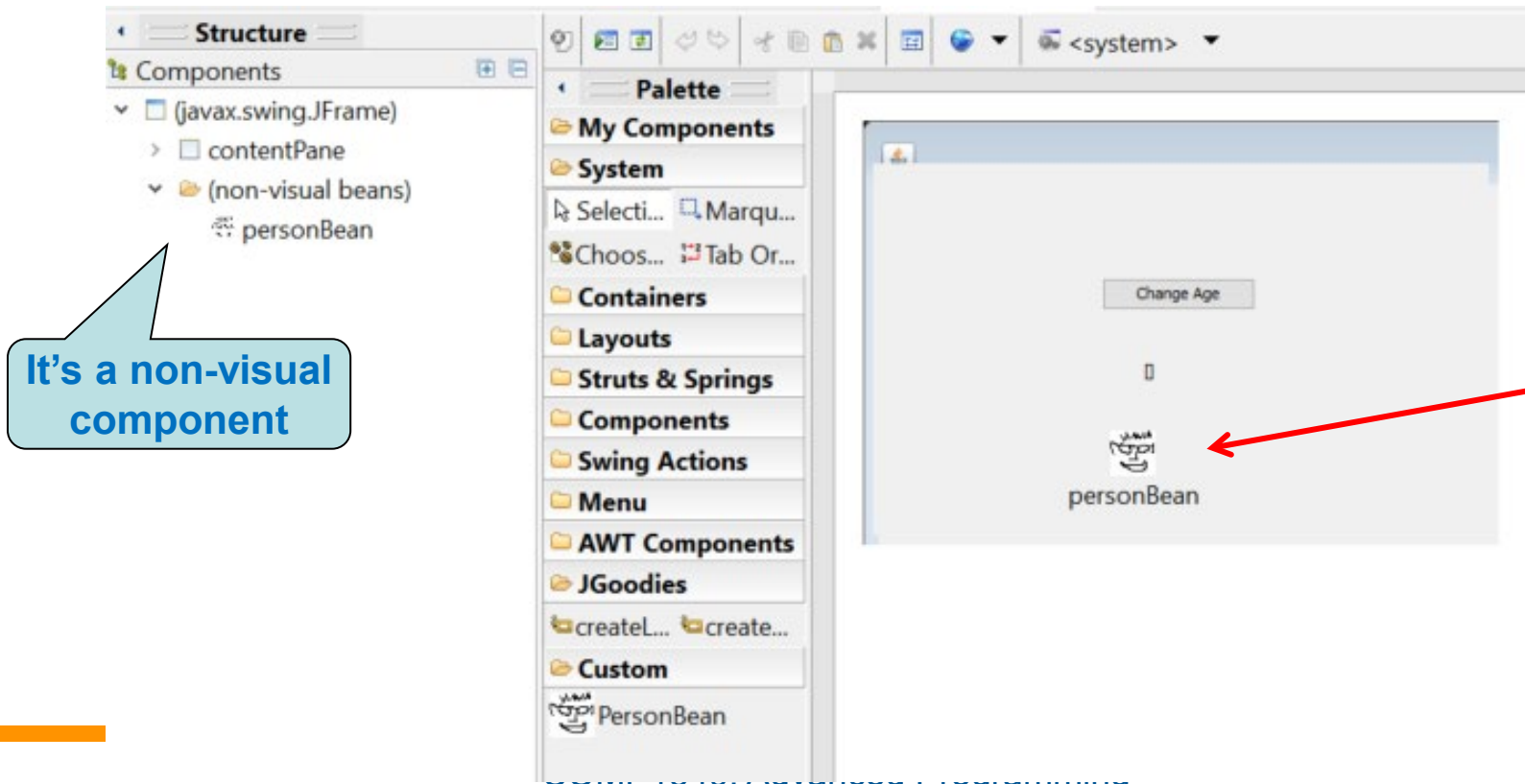
# Adding the PersonBean to the Palette

- The PersonBean component is now on the Palette



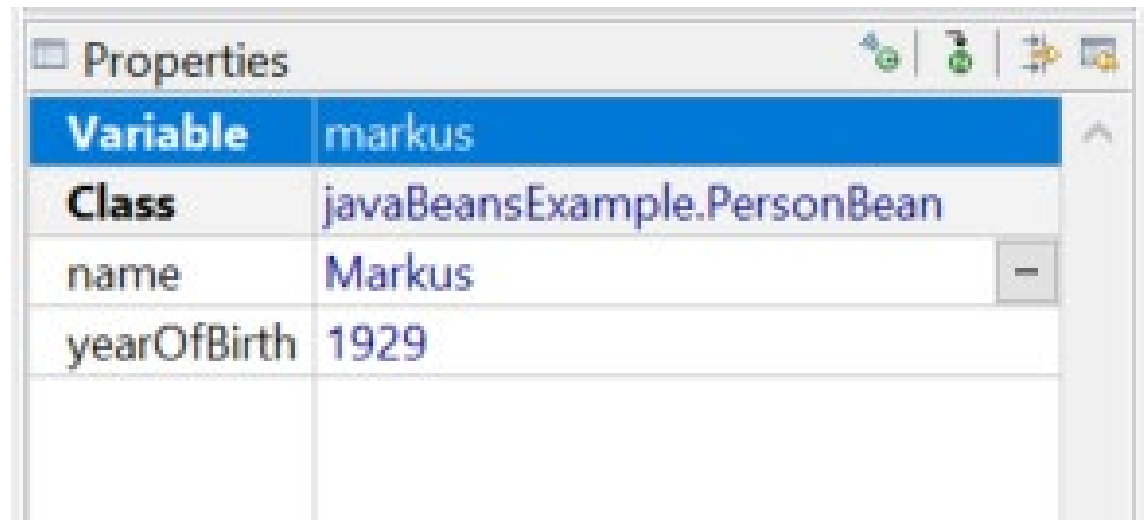
# Adding the PersonBean to the Palette

- The PersonBean can be dragged and dropped onto a form



# Setting Properties

- With the PersonBean component selected on the form, you can set the properties



# Generated Code

- Eclipse automatically generates the code based on how we set the properties

```
/**
 * Create the frame.
 */
public MainGUI() {
    markus.setYearOfBirth(1929);
    markus.setName("Markus");
    // ...
}
```

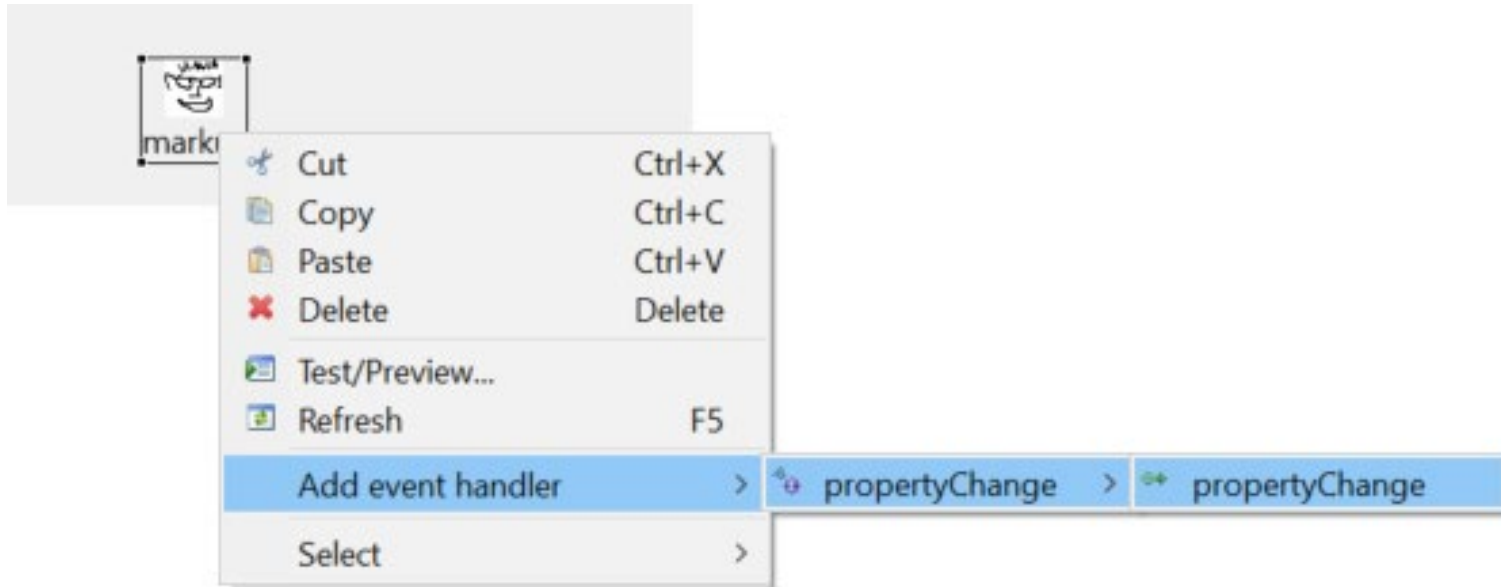
# Button Event

- We need to double-click the button and add code to the event handler to increment the year of birth

```
JButton btnChangeAge = new JButton("Change Age");  
btnChangeAge.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent arg0) {  
        markus.setYearOfBirth(markus.getYearOfBirth() + 1);  
    }  
});
```

# Linking Events

- It is also possible to link up events supported by the PersonBean and automatically generate a method to respond to the events



- The event handling method is generated and we just need to add code to update the label:

```
markus.addPropertyChangeListener(new PropertyChangeListener() {  
    public void propertyChange(PropertyChangeEvent evt) {  
        lblOutput.setText(evt.getSource().toString());  
    }  
});
```

You might have to move the generated method further down to ensure that the label has been declared

# How Does it Work

---

- Beans are able to expose their properties through a discovery process called Introspection. This relies on Reflection.

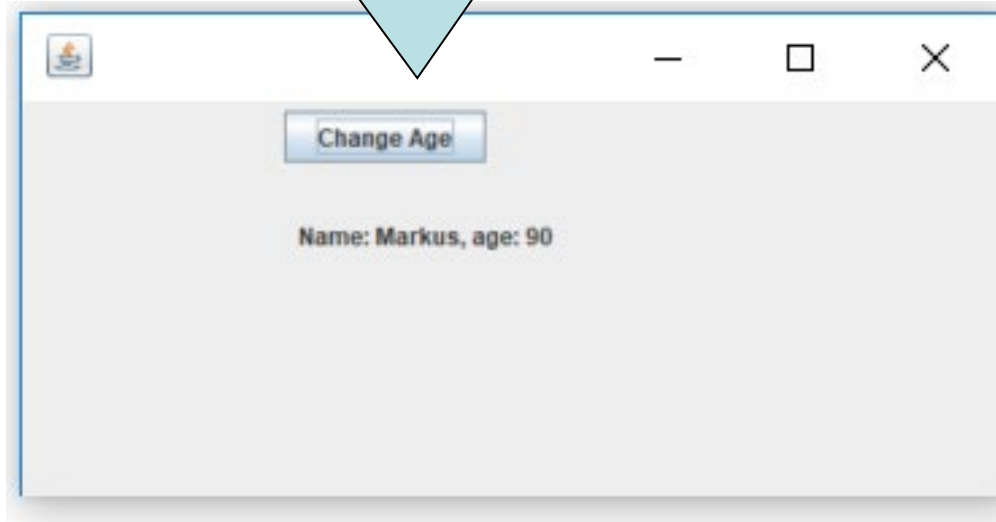
# Reflection

---

- Using reflection it is possible to discover the names of every method supported by the class
- If the name meets the JavaBeans specification it will be revealed during Introspection
- More on reflection in a future lecture

# The application running

Whenever the button is clicked, the property of the bean is changed which fires a property change event, which refreshes the label



# End of week 9!